

# Equations for the Self-Regulated Swarm and Migrating Flock Model

Generated from uploaded code

2026-06-08

## Contents

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| <b>1</b>  | <b>State Variables</b>                                 | <b>2</b>  |
| <b>2</b>  | <b>Neighbour Graph</b>                                 | <b>3</b>  |
| <b>3</b>  | <b>Fast Perceptual Bloch Dynamics</b>                  | <b>3</b>  |
| <b>4</b>  | <b>Bloch-to-Heading Alignment Weight</b>               | <b>5</b>  |
| <b>5</b>  | <b>Steering Cue Vector</b>                             | <b>5</b>  |
| <b>6</b>  | <b>Components of the Base Steering Vector</b>          | <b>6</b>  |
| 6.1       | Wall Repulsion . . . . .                               | 6         |
| 6.2       | Altitude Band for Drones . . . . .                     | 6         |
| 6.3       | Obstacle Avoidance . . . . .                           | 7         |
| 6.4       | Short-Range Separation . . . . .                       | 7         |
| 6.5       | Open-Domain Cohesion . . . . .                         | 8         |
| 6.6       | Adaptive Caution Gate . . . . .                        | 8         |
| <b>7</b>  | <b>Heading Noise and Bounded Turning</b>               | <b>8</b>  |
| <b>8</b>  | <b>Slow Regulatory Dynamics</b>                        | <b>9</b>  |
| <b>9</b>  | <b>Position Dynamics</b>                               | <b>10</b> |
| <b>10</b> | <b>Boundary Conditions</b>                             | <b>10</b> |
| 10.1      | Periodic Boundary . . . . .                            | 10        |
| 10.2      | Reflective Boundary . . . . .                          | 10        |
| 10.3      | Soft/Open Boundary in the Base Class . . . . .         | 10        |
| <b>11</b> | <b>Migrating Flock Extension</b>                       | <b>11</b> |
| 11.1      | Migratory Pull . . . . .                               | 11        |
| 11.2      | Lateral Corridor Confinement . . . . .                 | 11        |
| 11.3      | Total Steering Vector in the Migrating Flock . . . . . | 12        |
| <b>12</b> | <b>Streaming Obstacles in the Migration Script</b>     | <b>12</b> |

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>13 Observables and Diagnostics</b>                        | <b>13</b> |
| 13.1 Global Polar Order . . . . .                            | 13        |
| 13.2 Mean Regulatory Tone . . . . .                          | 13        |
| 13.3 Centroid . . . . .                                      | 13        |
| 13.4 Local Coherence . . . . .                               | 13        |
| 13.5 Largest Cluster Fraction . . . . .                      | 14        |
| 13.6 Width of the Flock . . . . .                            | 14        |
| <b>14 Full Algorithmic Update for One Step</b>               | <b>14</b> |
| <b>15 Parameter Values Used in the Migrating Drone Build</b> | <b>15</b> |
| <b>16 Important Implementation Caveat</b>                    | <b>16</b> |
| <b>17 Compact Full Model</b>                                 | <b>16</b> |

This note systematizes the equations implemented across two scripts:

1. `swarm_nonperiodic.py`: the base self-regulated swarm model with fast Bloch/GKSL-like perceptual channels, slow regulatory memory, steering cues, and boundary handling.
2. `drone_migration_obstacles.py`: the migrating 3D flock extension that adds a fixed migratory heading, a lateral flyway corridor, streaming obstacles, and coherence diagnostics.

The model is best understood as a fixed-speed collective-motion system whose heading dynamics is controlled by internal Bloch-like perceptual registers and a slow self-regulatory variable.

---

## 1 State Variables

For each agent/drone  $i = 1, \dots, N$ :

- Position:

$$\mathbf{x}_i(t) \in \mathbb{R}^d, \quad d \in \{2, 3\}.$$

- Unit heading vector:

$$\mathbf{e}_i(t) \in S^{d-1}, \quad \|\mathbf{e}_i(t)\| = 1.$$

- Slow regulatory variable:

$$s_i(t) \in \mathbb{R}.$$

- Fast perceptual Bloch channel from agent  $i$  to neighbour  $j$ :

$$\mathbf{m}_{ij}(t) = (m_x^{ij}(t), m_y^{ij}(t), m_z^{ij}(t)).$$

Here  $m_z^{ij} \in [-1, 1]$  is interpreted as the resolved tendency of agent  $i$  to align with neighbour  $j$ .

## 2 Neighbour Graph

The relative displacement from agent  $j$  to agent  $i$  is

$$\mathbf{d}_{ij} = \mathbf{x}_i - \mathbf{x}_j.$$

The Euclidean distance is

$$r_{ij} = \|\mathbf{d}_{ij}\|.$$

The neighbour adjacency matrix is

$$A_{ij} = \begin{cases} 1, & 0 < r_{ij} < R, \\ 0, & \text{otherwise.} \end{cases}$$

For non-periodic domains,  $\mathbf{d}_{ij} = \mathbf{x}_i - \mathbf{x}_j$ . For periodic domains, the minimum-image convention is used:

$$\mathbf{d}_{ij} = \mathbf{x}_i - \mathbf{x}_j - \mathbf{L} \text{round}\left(\frac{\mathbf{x}_i - \mathbf{x}_j}{\mathbf{L}}\right),$$

where  $\mathbf{L}$  is the domain-size vector.

The neighbour count of agent  $i$  is

$$n_i = \sum_j A_{ij}.$$


---

## 3 Fast Perceptual Bloch Dynamics

Each directed neighbour channel  $(i, j)$  has an internal Bloch vector

$$\mathbf{m}_{ij} = (m_x^{ij}, m_y^{ij}, m_z^{ij}).$$

The effective perceptual field is

$$\mathbf{h}_{ij} = (h_x, h_y, h_z^{ij}),$$

with

$$h_x = \Gamma, \quad h_y = 0,$$

and

$$h_z^{ij} = \kappa s_i + g_{\text{align}}(\mathbf{e}_i \cdot \mathbf{e}_j) + h_{\text{ext}}^{ij}.$$

The terms are:

- $\kappa s_i$ : slow-to-fast self-regulatory gain.
- $g_{\text{align}}(\mathbf{e}_i \cdot \mathbf{e}_j)$ : heading-consistency or persistence term.
- $h_{\text{ext}}^{ij}$ : optional external drive.

The field is masked by the neighbour graph:

$$h_z^{ij} = 0 \quad \text{if} \quad A_{ij} = 0.$$

The longitudinal equilibrium is

$$m_{z,\text{eq}}^{ij} = \tanh(h_z^{ij}).$$

The implemented driven-dissipative Bloch equations are

$$\begin{aligned} \dot{m}_x^{ij} &= -2h_z^{ij}m_y^{ij} - \frac{m_x^{ij}}{T_2}, \\ \dot{m}_y^{ij} &= 2(h_z^{ij}m_x^{ij} - \Gamma m_z^{ij}) - \frac{m_y^{ij}}{T_2}, \\ \dot{m}_z^{ij} &= 2\Gamma m_y^{ij} - \frac{m_z^{ij} - \tanh(h_z^{ij})}{T_1}. \end{aligned}$$

Equivalently, the coherent part can be read as a Bloch precession term

$$\dot{\mathbf{m}}_{ij}|_{\text{coh}} = 2\mathbf{h}_{ij} \times \mathbf{m}_{ij},$$

with dissipative relaxation

$$\begin{aligned} \dot{m}_x^{ij}|_{\text{diss}} &= -\frac{m_x^{ij}}{T_2}, & \dot{m}_y^{ij}|_{\text{diss}} &= -\frac{m_y^{ij}}{T_2}, \\ \dot{m}_z^{ij}|_{\text{diss}} &= -\frac{m_z^{ij} - m_{z,\text{eq}}^{ij}}{T_1}. \end{aligned}$$

The code uses explicit Euler updates:

$$m_a^{ij}(t + \Delta t) = m_a^{ij}(t) + \Delta t \dot{m}_a^{ij}(t), \quad a \in \{x, y, z\}.$$

For non-neighbouring pairs, the channel is reset to zero:

$$\mathbf{m}_{ij} = \mathbf{0} \quad \text{if} \quad A_{ij} = 0.$$

The longitudinal component is clipped:

$$m_z^{ij} \in [-1, 1].$$


---

## 4 Bloch-to-Heading Alignment Weight

The resolved longitudinal component  $m_z^{ij}$  is converted into an alignment weight

$$w_{ij} = \frac{m_z^{ij} + 1}{2}.$$

Thus

$$w_{ij} \in [0, 1].$$

The raw social heading drive is

$$\mathbf{D}_i^{\text{align}} = \sum_j A_{ij} w_{ij} \mathbf{e}_j.$$

In matrix form, if  $W = (w_{ij})$ , the code computes

$$\mathbf{D}^{\text{align}} = W\mathbf{E},$$

where  $\mathbf{E}$  is the matrix of agent headings.

---

## 5 Steering Cue Vector

The base model adds a steering vector

$$\mathbf{A}_i(\mathbf{X}, s_i)$$

to the social alignment drive. This vector contains wall avoidance, obstacle avoidance, inter-agent separation, altitude-band correction, and open-domain cohesion.

The total desired direction before noise is

$$\mathbf{D}_i = \mathbf{D}_i^{\text{align}} + \mathbf{A}_i(\mathbf{X}, s_i).$$

If  $\|\mathbf{D}_i\| \approx 0$ , the agent keeps its current heading:

$$\mathbf{D}_i = \mathbf{e}_i.$$

The target direction is then

$$\hat{\mathbf{D}}_i = \frac{\mathbf{D}_i}{\|\mathbf{D}_i\|}.$$


---

## 6 Components of the Base Steering Vector

The code constructs a preliminary avoidance/cohesion vector  $\mathbf{a}_i$ , then gates it by the slow regulatory tone.

### 6.1 Wall Repulsion

For soft or reflective boundaries, each coordinate  $k$  has lower and upper wall forces.

Lower-wall force:

$$f_{i,k}^{\text{lo}} = g_{\text{wall}} \left[ 1 - \frac{x_{i,k}}{M_{\text{wall}}} \right]_+.$$

Upper-wall force:

$$f_{i,k}^{\text{hi}} = g_{\text{wall}} \left[ 1 - \frac{L_k - x_{i,k}}{M_{\text{wall}}} \right]_+.$$

The contribution to coordinate  $k$  is

$$a_{i,k}^{\text{wall}} = \begin{cases} +f_{i,k}^{\text{lo}}, & x_{i,k} < M_{\text{wall}}, \\ -f_{i,k}^{\text{hi}}, & x_{i,k} > L_k - M_{\text{wall}}, \\ 0, & \text{otherwise.} \end{cases}$$

Here  $[z]_+ = \max(z, 0)$ ,  $L_k$  is the box size in coordinate  $k$ , and  $M_{\text{wall}}$  is the wall margin.

### 6.2 Altitude Band for Drones

For  $d = 3$ , if an altitude band

$$z_{\min} \leq z_i \leq z_{\max}$$

is specified, the vertical correction is

$$a_{i,z}^{\text{alt}} = \begin{cases} g_{\text{alt}}(z_{\min} - z_i), & z_i < z_{\min}, \\ -g_{\text{alt}}(z_i - z_{\max}), & z_i > z_{\max}, \\ 0, & z_{\min} \leq z_i \leq z_{\max}. \end{cases}$$

### 6.3 Obstacle Avoidance

For a circular or spherical obstacle with centre  $\mathbf{c}_q$  and radius  $\rho_q$ , define

$$\mathbf{r}_{iq} = \mathbf{x}_i - \mathbf{c}_q, \quad r_{iq} = \|\mathbf{r}_{iq}\|.$$

If

$$r_{iq} < \rho_q + M_{\text{wall}},$$

the obstacle repulsion strength is

$$\alpha_{iq} = g_{\text{wall}} \left[ 1 - \frac{r_{iq} - \rho_q}{M_{\text{wall}}} \right]_+.$$

The obstacle contribution is

$$\mathbf{a}_i^{\text{obs}} = \sum_q \alpha_{iq} \frac{\mathbf{x}_i - \mathbf{c}_q}{\|\mathbf{x}_i - \mathbf{c}_q\|}.$$

### 6.4 Short-Range Separation

If two agents are neighbours and too close,

$$A_{ij} = 1, \quad r_{ij} < r_{\text{sep}},$$

the separation weight is

$$\beta_{ij} = g_{\text{sep}} \left( 1 - \frac{r_{ij}}{r_{\text{sep}}} \right).$$

The separation contribution is

$$\mathbf{a}_i^{\text{sep}} = \sum_j \beta_{ij} \frac{\mathbf{x}_i - \mathbf{x}_j}{\|\mathbf{x}_i - \mathbf{x}_j\|}.$$

## 6.5 Open-Domain Cohesion

For open boundary mode, let the local centre of mass be

$$\mathbf{c}_i^{\text{local}} = \frac{1}{n_i} \sum_{j: A_{ij}=1} \mathbf{x}_j, \quad n_i > 0.$$

The cohesion contribution is

$$\mathbf{a}_i^{\text{coh}} = g_{\text{coh}}(\mathbf{c}_i^{\text{local}} - \mathbf{x}_i).$$

## 6.6 Adaptive Caution Gate

The preliminary steering vector is

$$\mathbf{a}_i = \mathbf{a}_i^{\text{wall}} + \mathbf{a}_i^{\text{alt}} + \mathbf{a}_i^{\text{obs}} + \mathbf{a}_i^{\text{sep}} + \mathbf{a}_i^{\text{coh}}.$$

The slow regulatory variable gates this vector by

$$G_i(s_i) = 1 + c_{\text{caution}}\sigma(s_i),$$

where

$$\sigma(s_i) = \frac{1}{1 + e^{-s_i}}.$$

Thus the base steering vector returned by `SelfRegulatedSwarm._avoidance_vector` is

$$\mathbf{A}_i^{\text{base}} = G_i(s_i) \mathbf{a}_i.$$


---

## 7 Heading Noise and Bounded Turning

A random noise vector is added to the target direction:

$$\tilde{\mathbf{D}}_i = \text{Normalize}(\hat{\mathbf{D}}_i + \xi_i),$$

where

$$\xi_i \sim \mathcal{N}(0, \eta^2 I_d).$$

The heading does not jump instantly to  $\tilde{\mathbf{D}}_i$ . Instead, it rotates toward it by at most

$$\Delta\theta_{\text{max}} = \omega_{\text{max}}\Delta t.$$



Let

$$\theta_i = \arccos(\mathbf{e}_i \cdot \tilde{\mathbf{D}}_i),$$

and

$$\delta_i = \min(\theta_i, \omega_{\max} \Delta t).$$

Define the unit perpendicular direction

$$\mathbf{u}_i = \frac{\tilde{\mathbf{D}}_i - (\mathbf{e}_i \cdot \tilde{\mathbf{D}}_i) \mathbf{e}_i}{\|\tilde{\mathbf{D}}_i - (\mathbf{e}_i \cdot \tilde{\mathbf{D}}_i) \mathbf{e}_i\|}.$$

Then the bounded-turn update is

$$\mathbf{e}_i(t + \Delta t) = \text{Normalize}(\cos \delta_i \mathbf{e}_i(t) + \sin \delta_i \mathbf{u}_i).$$


---

## 8 Slow Regulatory Dynamics

The mean resolved alignment received by agent  $i$  is

$$M_i = \frac{1}{\max(n_i, 1)} \sum_j A_{ij} m_z^{ij}.$$

The slow target is

$$s_i^{\text{target}} = \tanh(\lambda_{\text{fb}} M_i).$$

The implemented slow memory dynamics is

$$\dot{s}_i = -\gamma_s (s_i - s_{\text{base}}) + \gamma_s s_i^{\text{target}}.$$

Equivalently,

$$\dot{s}_i = \gamma_s [-(s_i - s_{\text{base}}) + \tanh(\lambda_{\text{fb}} M_i)].$$

The explicit Euler update is

$$s_i(t + \Delta t) = s_i(t) + \Delta t \gamma_s [-(s_i(t) - s_{\text{base}}) + \tanh(\lambda_{\text{fb}} M_i(t))].$$

The memory horizon is approximately

$$\tau_s \sim \frac{1}{\gamma_s}.$$


---

## 9 Position Dynamics

The physical motion is fixed-speed advection along the current heading:

$$\dot{\mathbf{x}}_i = v_0 \mathbf{e}_i.$$

The explicit Euler update is

$$\mathbf{x}_i(t + \Delta t) = \mathbf{x}_i(t) + \Delta t v_0 \mathbf{e}_i(t + \Delta t).$$

The code updates heading first and then updates position using the new heading.

---

## 10 Boundary Conditions

### 10.1 Periodic Boundary

$$\mathbf{x}_i \leftarrow \mathbf{x}_i \bmod \mathbf{L}.$$

### 10.2 Reflective Boundary

If coordinate  $x_{i,k}$  crosses the lower wall,

$$x_{i,k} \leftarrow -x_{i,k}, \quad e_{i,k} \leftarrow -e_{i,k}.$$

If it crosses the upper wall,

$$x_{i,k} \leftarrow 2L_k - x_{i,k}, \quad e_{i,k} \leftarrow -e_{i,k}.$$

The heading is renormalized after reflection.

### 10.3 Soft/Open Boundary in the Base Class

For `soft` and `open` modes in the base class, the position is clipped as a safety net:

$$\mathbf{x}_i \leftarrow \text{clip}(\mathbf{x}_i, 0, \mathbf{L}).$$

In the migrating-flock subclass, this boundary application is overridden by `pass`, so the flock can travel freely in the migration direction.

---

## 11 Migrating Flock Extension

The class `MigratingFlock` modifies the steering vector by adding a constant migratory pull and lateral corridor confinement.

The migration direction is

$$\hat{\mathbf{m}} = \text{Normalize}(\mathbf{m}_{\text{dir}}).$$

In the provided build configuration,

$$\mathbf{m}_{\text{dir}} = (0, 1, 0),$$

so the flock migrates in the positive  $y$ -direction.

### 11.1 Migratory Pull

The migratory steering contribution is

$$\mathbf{A}_i^{\text{mig}} = g_{\text{mig}}(1 + n_i)\hat{\mathbf{m}}.$$

The factor  $(1 + n_i)$  scales the migratory pull to remain comparable with the neighbour-alignment sum.

### 11.2 Lateral Corridor Confinement

The flyway corridor is imposed along the lateral  $x$ -coordinate. Let

$$L_c > 0$$

be the corridor half-width. The always-on centering spring is

$$A_{i,x}^{\text{spring}} = -k_{\text{center}}x_i.$$

Define the corridor excess

$$q_i = |x_i| - L_c.$$

The stiff wall term is

$$A_{i,x}^{\text{corridor-wall}} = \begin{cases} -\text{sign}(x_i)g_{\text{conf}}(1 + q_i), & q_i > 0, \\ 0, & q_i \leq 0. \end{cases}$$

Thus the total lateral corridor force is

$$A_{i,x}^{\text{corridor}} = -k_{\text{center}}x_i - \text{sign}(x_i)g_{\text{conf}}(1 + |x_i| - L_c)\mathbf{1}_{|x_i| > L_c}.$$

### 11.3 Total Steering Vector in the Migrating Flock

The migrating-flock steering vector is

$$\mathbf{A}_i^{\text{migrating}} = \mathbf{A}_i^{\text{base}} + \mathbf{A}_i^{\text{mig}} + \mathbf{A}_i^{\text{corridor}}.$$

Therefore, the full desired heading drive becomes

$$\mathbf{D}_i = \sum_j A_{ij} \frac{m_z^{ij} + 1}{2} \mathbf{e}_j + \mathbf{A}_i^{\text{base}} + \mathbf{A}_i^{\text{mig}} + \mathbf{A}_i^{\text{corridor}}.$$


---

## 12 Streaming Obstacles in the Migration Script

The obstacle stream is not a differential equation but a procedural environment update. Obstacles behind the flock are removed:

$$y_q \leq y_{\text{centroid}} - B \implies \text{remove obstacle } q,$$

where  $B$  is the behind distance.

New obstacles are generated ahead of the flock until

$$y_{\text{front}} \geq y_{\text{centroid}} + A,$$

where  $A$  is the ahead distance.

The longitudinal gap between obstacle rows is sampled as

$$\Delta y \sim U(g_{\min}, g_{\max}).$$

Obstacle lateral position is sampled as

$$x_q \sim U(-L_c + 1, L_c - 1),$$

and the altitude coordinate is sampled inside the altitude band:

$$z_q \sim U(z_{\min}, z_{\max}).$$

Obstacle radius is sampled as

$$\rho_q \sim U(\rho_{\min}, \rho_{\max}).$$


---

## 13 Observables and Diagnostics

### 13.1 Global Polar Order

The global polar order is

$$P(t) = \left\| \frac{1}{N} \sum_{i=1}^N \mathbf{e}_i(t) \right\|.$$

This is returned as `polar`.

### 13.2 Mean Regulatory Tone

The mean slow regulatory tone is

$$S(t) = \frac{1}{N} \sum_{i=1}^N s_i(t).$$

This is returned as `tone`.

### 13.3 Centroid

The flock centroid is

$$\mathbf{C}(t) = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i(t).$$

The migration distance shown in the animation is essentially  $C_y(t)$ .

### 13.4 Local Coherence

For each agent with at least one neighbour,

$$\bar{\mathbf{e}}_i = \frac{1}{n_i} \sum_{j: A_{ij}=1} \mathbf{e}_j.$$

The local coherence is

$$C_{\text{local}} = \frac{1}{|\mathcal{J}|} \sum_{i \in \mathcal{J}} \|\bar{\mathbf{e}}_i\|,$$

where

$$\mathcal{J} = \{i : n_i > 0\}.$$

If no agent has neighbours, the code returns

$$C_{\text{local}} = 1.$$

### 13.5 Largest Cluster Fraction

Construct the undirected graph induced by the sensing-radius adjacency relation. Let  $K_{\text{max}}$  be the number of agents in the largest connected component. The largest-cluster fraction is

$$C_{\text{cluster}} = \frac{K_{\text{max}}}{N}.$$

A value near 1 means the flock is spatially whole. A lower value indicates splitting around obstacles.

### 13.6 Width of the Flock

The lateral width used in the headless report is

$$W_x(t) = \text{std}\{x_1, x_2, \dots, x_N\}.$$


---

## 14 Full Algorithmic Update for One Step

For each timestep  $t \rightarrow t + \Delta t$ :

1. Compute neighbour graph:

$$A_{ij} = \mathbf{1}_{0 < \|\mathbf{x}_i - \mathbf{x}_j\| < R}.$$

2. Compute heading consistency:

$$C_{ij} = \mathbf{e}_i \cdot \mathbf{e}_j.$$

3. Compute longitudinal field:

$$h_z^{ij} = \kappa s_i + g_{\text{align}} C_{ij} + h_{\text{ext}}^{ij}.$$

4. Apply neighbour mask:

$$h_z^{ij} \leftarrow A_{ij} h_z^{ij}.$$

5. Update fast Bloch variables:

$$(m_x^{ij}, m_y^{ij}, m_z^{ij}) \leftarrow (m_x^{ij}, m_y^{ij}, m_z^{ij}) + \Delta t (\dot{m}_x^{ij}, \dot{m}_y^{ij}, \dot{m}_z^{ij}).$$

6. Convert  $m_z^{ij}$  to alignment weights:

$$w_{ij} = \frac{m_z^{ij} + 1}{2}.$$

7. Build desired heading vector:

$$\mathbf{D}_i = \sum_j A_{ij} w_{ij} \mathbf{e}_j + \mathbf{A}_i(\mathbf{X}, s_i).$$

8. Add heading noise and normalize.

9. Rotate  $\mathbf{e}_i$  toward the noisy target by at most  $\omega_{\max} \Delta t$ .

10. Update the slow memory:

$$s_i \leftarrow s_i + \Delta t \gamma_s [-(s_i - s_{\text{base}}) + \tanh(\lambda_{\text{fb}} M_i)].$$

11. Move the agent:

$$\mathbf{x}_i \leftarrow \mathbf{x}_i + \Delta t v_0 \mathbf{e}_i.$$

12. Apply boundary rules, unless overridden by the migrating-flock subclass.

---

## 15 Parameter Values Used in the Migrating Drone Build

The `build()` function in `drone_migration_obstacles.py` uses:

$$N = 120, \quad d = 3, \quad v_0 = 2.0, \quad R = 9.0.$$

Altitude band:

$$z_{\min} = 10.0, \quad z_{\max} = 18.0, \quad g_{\text{alt}} = 3.0.$$

Actuation and noise:

$$\omega_{\max} = 0.40, \quad \eta = 0.10.$$

Cohesion and separation:

$$g_{\text{coh}} = 0.35, \quad g_{\text{sep}} = 12.0, \quad r_{\text{sep}} = 1.8.$$

Obstacle repulsion:

$$g_{\text{wall}} = 16.0, \quad M_{\text{wall}} = 8.0.$$

Slow memory:

$$\gamma_s = 0.025.$$

Migration:

$$\hat{\mathbf{m}} = (0, 1, 0), \quad g_{\text{mig}} = 0.45.$$

Corridor:

$$L_c = 16.0, \quad g_{\text{conf}} = 1.5, \quad k_{\text{center}} = 0.12.$$

Adaptive caution:

$$c_{\text{caution}} = 0.6.$$

---

## 16 Important Implementation Caveat

The comments in the source describe walls, obstacles, separation, altitude, cohesion, and migration as external-drive channels  $P(X(t))$ . In the executable implementation, however, these cues are mostly added after the Bloch update as steering vectors in the heading map:

$$\mathbf{D}_i = \sum_j A_{ij} w_{ij} \mathbf{e}_j + \mathbf{A}_i(\mathbf{X}, s_i).$$

The optional argument `h_ext` can inject an external drive directly into

$$h_z^{ij},$$

but the migrating-obstacle script does not explicitly pass such an `h_ext` matrix during `sim.step()`. Thus, in the current code, obstacle avoidance and migration affect the heading map directly, while the Bloch channel mainly controls neighbour-alignment weights.

---

## 17 Compact Full Model

The model can be summarized as

$$\dot{\mathbf{x}}_i = v_0 \mathbf{e}_i$$

with heading controlled by

$$\mathbf{e}_i \rightarrow \text{RotateToward}_{\omega_{\max} \Delta t} \left[ \mathbf{e}_i, \text{Normalize} \left( \sum_j A_{ij} \frac{m_z^{ij} + 1}{2} \mathbf{e}_j + \mathbf{A}_i(\mathbf{X}, s_i) + \xi_i \right) \right]$$



where

$$\begin{aligned}\dot{m}_x^{ij} &= -2h_z^{ij}m_y^{ij} - \frac{m_x^{ij}}{T_2}, \\ \dot{m}_y^{ij} &= 2(h_z^{ij}m_x^{ij} - \Gamma m_z^{ij}) - \frac{m_y^{ij}}{T_2}, \\ \dot{m}_z^{ij} &= 2\Gamma m_y^{ij} - \frac{m_z^{ij} - \tanh(h_z^{ij})}{T_1},\end{aligned}$$

and

$$h_z^{ij} = \kappa s_i + g_{\text{align}}(\mathbf{e}_i \cdot \mathbf{e}_j) + h_{\text{ext}}^{ij}$$

with slow feedback

$$\dot{s}_i = \gamma_s [-(s_i - s_{\text{base}}) + \tanh(\lambda_{\text{fb}} M_i)]$$

and

$$M_i = \frac{1}{\max(n_i, 1)} \sum_j A_{ij} m_z^{ij}.$$